

Lab 6: Functions and File Handling

Remarks

- Exercises L6.E1 and L6.E2 in this lab session will be **evaluated**.
- First practice the programs mentioned in L6.1 and L7.1. These will not be evaluated, and you do not have to submit them.
- You **must** draw and submit **flowcharts** to express the algorithms of each of your programs. If you **do not submit**, then the submission will be incomplete and **will not be accepted**.
- All the source codes and diagrams related to the Exercises L6.E1 and L6.E2 should be submitted to the respective lab activity on Moodle.

L6.1 Practice Programs

1. For basic understanding, practice the programs in Chapters 9 and 10 of the CS1033 Course Notes.

L6.2 Exercises

Exercise L6.E1 - Develop a program to read in two matrices A and B with dimensions $n \times m$ and compute and display their product AB^T (of dimensions $n \times n$). Assume that the elements of the matrices are integers. You must use functions while implementing this program. You must draw a flowchart to express the algorithm of your program. An example is shown below.

Note: You should check for any errors when entering the matrices. Print “**Invalid Matrix**” as the error message if the entered matrix does not comply with the given dimensions. Print “**Error**” for any other exceptions you are handling.

```
$ python matrix.py
Enter the dimension: 3,4
Enter Matrix A:
1 2 3 4
3 3 4 4
4 4 5 5

Enter Matrix B:
1 7 3 3
3 7 4 4
5 7 5 5

Matrix A X Transpose(B) :
36 45 54
48 62 76
62 80 98
```

Input: A line corresponds to a row of a matrix. The elements in a row will be separated by a single whitespace. The output should be formatted similarly.

Define global variables of two-dimensional arrays to store the values of the matrices A and B. Break down the program into four functions as follows:

1. Read in the elements of the matrices A and B
2. Compute the Transpose of a matrix
3. Compute the product of the two matrices

4. Display the resultant matrix AB^T

Exercise L6.E2 – Develop a program, using the provided skeleton code, to read the name, birthday, and gender of a person from a file and output the ten-digit national identity card (NIC) number. The first four digits of the ID card is the birth year, and the next three digits are the number of days to the birth date from January 1st of that year. If the person is a female, 500 is added to that value. The next 3 digits are assigned in the order of submission for a particular birth year (The input file contains the records in the order of submission where each attribute is space separated). You must draw a flowchart to express the algorithm of your program. An Example is shown below.

Input File:

```
Saman 1990-05-03 M
Aruni 1990-04-06 F
Kumaran 1988-03-05 M
Nazar 1997-09-24 M
```

Output File :

```
Saman 1990123001
Aruni 1990596002
Kumaran 1988065001
Nazar 1997267001
```

Note:

1. A skeleton code is provided to you on Moodle to complete this Lab activity.
2. Develop the program using the given skeleton.

Lab 7: Beams and Cars

Remarks

- This lab session will be **evaluated**.
- You **must** draw and submit **flowcharts** to express the algorithms of each of your programs. If you **do not submit**, then the submission will be incomplete and **will not be accepted**.
- All the source codes and diagrams related to the Exercises L7.E1 and L7.E2 should be submitted to the respective lab activity on Moodle.

L7.1 Practice Programs

1. No special practice programs are provided. Lab 6 must be completed before attempting this lab.

L7.2 Exercises

Exercise L7.E1 – You are given a file named beam_data.txt containing data about multiple beams. Each line in the file represents data for one beam, formatted as follows:

Length YoungsModulus MomentOfInertia Load

- Length: The length of the beam in meters.
- YoungsModulus: The Young's modulus of the material in GPa (Gigapascals).
- MomentOfInertia: The moment of inertia of the beam's cross-section in m^4 (meters to the fourth power).
- Load: The load applied to the center of the beam in kN (kilonewtons).

Example line:

5.0 210 0.0001 10

Example Input File (beam_data.txt):

```
5.0 210 0.0001 10
6.0 200 0.0002 12
4.0 190 0.00015 8
7.0 250 0.0001 15
8.0 220 0.00025 20
```

Develop a python program to read the file and calculate the following for each beam:

1. The maximum deflection at the center of the beam.
2. The maximum bending stress at the center of the beam.

Note: The following formulas can be used.

Maximum deflection (Dmax) :

$$Dmax = \frac{PL^3}{48EI}$$

where;

P - load in Newtons,

L - length,

E - Young's modulus,

I - moment of inertia.

Maximum bending stress (Smax)

$$Smax = \frac{PL}{4I}$$

Example Output Format:

```
Beam 1: Length: 5.0 m, Max Deflection: 0.000001 m, Max Bending
Stress: 250000000.00 Pa
Beam 2: Length: 6.0 m, Max Deflection: 0.000007 m, Max Bending
Stress: 150000000.00 Pa
Beam 3: Length: 4.0 m, Max Deflection: 0.000002 m, Max Bending
Stress: 133333333.33 Pa
Beam 4: Length: 7.0 m, Max Deflection: 0.000003 m, Max Bending
Stress: 375000000.00 Pa
Beam 5: Length: 8.0 m, Max Deflection: 0.000009 m, Max Bending
Stress: 200000000.00 Pa
```

Exercise L7.E2 – EcoMotion Autos is a car manufacturing company. They are conducting research regarding their 3 brand new car models that are going to be released next month. You have been requested to assist them by helping them to analyze the data taken during the test drive.

You are provided with three input files model1.txt, model2.txt and model3.txt containing the speed measured at the end of each time interval.

Example input files :

model1.txt

```
0      0
100    1
200    5
300    13
400    29
500    57
600    68
700    78
800    89
900    99
1000   111
```

model2.txt

```
0      0
100    1
200    7
300    12
400    31
500    63
600    74
700    89
800    98
900    107
1000   120
```

model3.txt

```
0      0
100    5
200    15
300    21
400    39
500    55
600    69
700    81
800    101
900    113
1000   122
```

- First column in each file is the time measured in milliseconds.
- Second column is the speed measured in kilometers per hour.

Note: The time values for every input file are the same. (0, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000)

In this lab, you need to develop a python program to calculate and output the maximum acceleration reached by each car model during the test drive using the provided skeleton code. Furthermore, you need to plot a graph of acceleration against time for each car model.

1. Complete the given “get_speed” function to read the speed values from a file, when the filename is given as the input. The function should output the speed values as a list of integers.

- The speed values are given in kilometers per hour format. Complete the “convert_kmph_to_ms” function to convert the speeds into meters per second when the file name is given as the input. Round off the values to 4 decimal places. Use the function implemented in the previous step to read the speeds from the file.

Formula to convert kilometers per hour into meters per second.

$$1 \text{ km/h} = \frac{1000 \text{ meters}}{3600 \text{ seconds}} = \frac{5}{18} \text{ m/s}$$

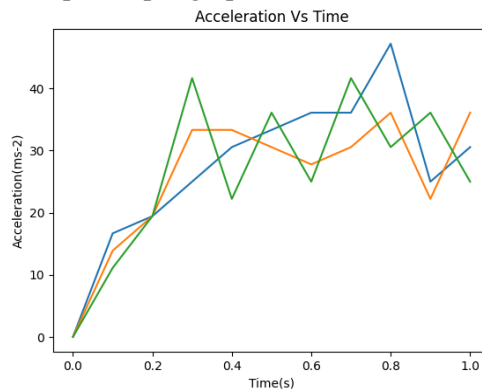
- Complete the “get_acceleration” function to get the acceleration of each model against the time. The function will output a list of floats. Round off the values to 2 decimal places. (Acceleration at the start is zero, consider the time interval as 100 milliseconds.)

Example acceleration calculations :

Time (milliseconds)	Speed (ms ⁻¹)	Acceleration (ms ⁻²)
0	0	0
100	10	(10-0)/0.1 = 100
200	20	(20-10)/0.1 = 100
300	24	(24-20)/0.1 = 40

- Plot a line graph for acceleration against time for each model. Plot the lines for all three models in the same graph.

Example output graph :



- Get the maximum acceleration reached by each model to a file named ‘max_acceleration.txt’ in the format given in the sample output.

Example output file (max_acceleration.txt) :

```
model1      12
model2      8
model3      9
```

Lab 8: Circuits and Matrices

Remarks

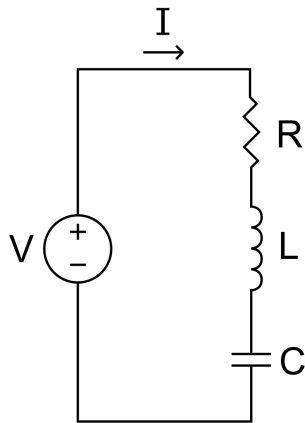
- This lab session will be **evaluated**.
- You **must** draw and submit **flowcharts** to express the algorithms of each of your programs. If you **do not submit**, then the submission will be incomplete and **will not be accepted**.
- All the source codes and diagrams related to the Exercise L8.E1 and L8.E2 should be submitted to the respective lab activity on Moodle.

L8.1 Practice Programs

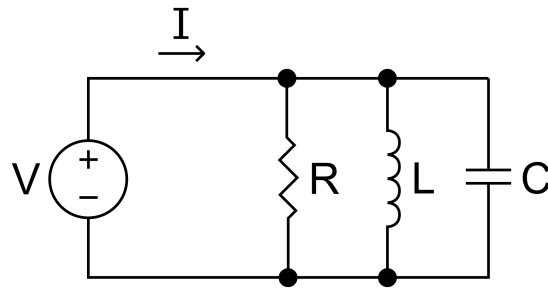
1. No special practice programs are provided. Lab 7 must be completed before attempting this lab.

L8.2 Exercises

Exercise L8.E1 – In this lab, you are required to develop a Python program to analyze R-L-C circuits. You must also draw a flowchart to express the algorithm of your program.



RLC Series Circuit



RLC Parallel Circuit

The equations required for the RLC circuit analysis are:

- Angular frequency (ω)

$$\omega = 2\pi f \quad \text{where } f \text{ is frequency}$$

- Magnitude of impedance of the inductor (Z_L)

$$Z_L = \omega L \quad \text{where } L \text{ is inductance of the inductor}$$

- Magnitude of impedance of the capacitor (Z_C)

$$Z_C = \frac{1}{\omega C} \quad \text{where } C \text{ is capacitance of the capacitor}$$

- Magnitude of total impedance of the circuit (Z_{total})
 - Series

$$Z_{total} = \sqrt{R^2 + (Z_L - Z_C)^2}$$

- Parallel

$$\frac{1}{Z_{total}} = \sqrt{\frac{1}{R^2} + \left(\frac{1}{Z_L} - \frac{1}{Z_C}\right)^2}$$

- Current through the circuit (I)

$$I = \frac{V}{Z_{total}} \quad \text{where V is voltage across the circuit as shown in the picture}$$

- Phase angle between voltage and current in **radian**.

- Series

$$\phi = \tan^{-1} \left(\frac{Z_L - Z_C}{R} \right)$$

- Parallel

$$\phi = \tan^{-1} \left(\left(\frac{1}{Z_L} - \frac{1}{Z_C} \right) \cdot R \right)$$

You will have an input text file to get the input values and you need to write the results to a text file named “result.txt”. The input and output format are as follows:

Each line in the input file will follow this format: **Type R L C V f**, where:

- **Type** specifies whether the circuit is "series" or "parallel".
- **R, L, C** are the values of resistance (in **ohms**), inductance (in **millihenries**), and capacitance (in **microfarads**), respectively.
- **V** is the voltage amplitude (in **volts**).
- **f** is the frequency (in **hertz**).

Example Input File (input.txt):

```
series 12 150 100 100 50
parallel 1000 142 160 240 60
parallel 50 20 5 50 100
```

For each line in the input file, output the following calculated values in **a single line separated by spaces**. Each line in the output file will follow this format: Z_L Z_C Z_{total} A ϕ

- Magnitude of impedance of the inductor in **ohms**.
- Magnitude of impedance of the capacitor in **ohms**.

- Magnitude of total impedance of the circuit in **ohms**.
- Current through the circuit in **Ampere (A)**.
- Phase angle between voltage and current in **degree**.

All the values should be rounded to **one decimal point**.

Example Output File (result.txt):

```
47.1 31.8 19.4 5.1 51.9
53.5 16.6 24.0 10.0 -88.6
12.6 318.3 12.7 4.0 75.3
```

Follow the instructions below.

1. Your program should take as input *the name of the text file* that contains the values of **Type R L C V f** in each line. Have a text file name (we can call it the input file) containing the values in the same folder where the source file has been saved. Your program should assume the input file exists and it is valid.
2. Develop a Python function named “analyseSeriesCircuit” for analyzing series R-L-C circuits and “analyseParallelCircuit” for analyzing parallel R-L-C circuits, which take **R L C V f** values as arguments and return the Z_L Z_C Z_{total} A ϕ values.
3. Write the above results to a text file named “result.txt”. This file should be in the same folder where the source file is.
4. The program should call appropriate functions to achieve the task.

Exercise L8.E2 – In this exercise, you are required to develop a Python program to find the inverse of a matrix.

If A is a non-singular $n \times n$ square matrix, there is an existence of $n \times n$ matrix A^{-1} , which is called the inverse matrix of A such that it satisfies the below property:

$$A A^{-1} = A^{-1} A = I, \quad \text{where } I \text{ is the Identity matrix}$$

- **Inverse of a 2x2 matrix:**

Let $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ be a 2x2 matrix.

The inverse matrix of A is given by the formula,

$$A^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

- **For higher order matrices: Inverse Matrix Formula**

$$A^{-1} = \frac{1}{|A|} \text{Adj}(A)$$

- $|A|$ is the determinant of A and $|A| \neq 0$

- $\text{Adj}(A)$ is the adjoint of A

- **Terms Related to Matrix Inverse**

Minor: Minor of matrix for a particular element in the matrix is defined as the matrix obtained after deleting the row and column of the matrix in which that particular element lies. The $(i,j)^{\text{th}}$ minor (M_{ij}) of a matrix is obtained by eliminating the i^{th} row and j^{th} column.

$$\text{Let matrix } A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$M_{11} = \begin{bmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{bmatrix} \quad M_{22} = \begin{bmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{bmatrix}$$

M_{ij} is the $(i,j)^{\text{th}}$ minor matrix

Cofactor: The cofactor(C_{ij}) is obtained by multiplying the determinant of the $(i,j)^{\text{th}}$ minor matrix with $(-1)^{i+j}$.

$$C_{ij} = (-1)^{i+j} \times \det(M_{ij})$$

Cofactor Matrix: The matrix formed with the cofactors of the elements of the matrix is called the cofactor matrix.

$$\text{Cofactor Matrix} = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix}$$

Adjoint of Matrix: The adjoint of a matrix is the transpose of the cofactor matrix.

$$\text{Adj}(A) = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix}^T$$

Determinant: Determinant of a matrix is equal to the summation of the product of the elements of a particular row or column with their respective cofactors.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$\det(A) = a_{11}(-1)^{1+1} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} + a_{12}(-1)^{1+2} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13}(-1)^{1+3} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix}$$

You will have an input text file named “matrix_data.txt” containing several $n \times n$ square matrices ($2 \leq n \leq 10$). Write a python program to calculate the inverse and display inverse of each matrix. Assume that the elements of the input matrices are integers and all of the input matrices are invertible, meaning they have a non-zero determinant and thus an inverse exists for each of them.

The “matrix_data.txt” text file is formatted as below.

1. The first line of the text file is an integer representing the number of matrices.
2. The first line of each matrix is an integer **n**, the size of the matrix.
3. The next **n** lines are composed of n integers separated by commas, representing each row of the matrix.

Example Input File (matrix_data.txt):

```
2
2
1, 2
3, 5
3
1, 2, 1
3, 2, 1
1, 0, 1
```

Output:

```
Inverse of Matrix 1:
-5.00    2.00
 3.00   -1.00
Inverse of Matrix 2:
-0.50    0.50    0.00
 0.50    0.00   -0.50
 0.50   -0.50    1.00
```

Note: When printing each row of the inverse matrix, use field width:7 and precision:2 to format each element of the row (example: `print("".join(f"{elem:7.2f}" for elem in row))`).

Break down the program into six functions as follows:

1. Read the matrix data from the input file.
2. Compute the Determinant of a matrix.
3. Get the Minor of a matrix.
4. Compute the Transpose of a matrix.
5. Compute the Inverse of a matrix.
6. Display the resultant inverse matrices.

Lab 9: Voltages and Chemicals

Remarks

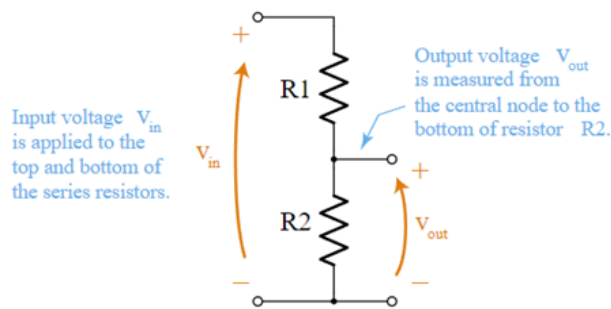
- This lab session will be **evaluated**.
- You **must** draw and submit **flowcharts** to express the algorithms of each of your programs. If you **do not submit**, then the submission will be incomplete and **will not be accepted**.
- All the source codes and diagrams related to the Exercises L9.E1 and L9.E2 should be submitted to the respective lab activity on Moodle.

L9.1 Practice Programs

1. No special practice programs are provided. Lab 8 must be completed before attempting this lab.

L9.2 Exercises

Exercise L9.E1 - A voltage divider (also known as a potential divider) is a passive linear circuit that produces an output voltage (V_{out}) that is a fraction of its input voltage (V_{in}). Voltage division results from distributing the input voltage among the components of the divider (In this case, Resistors).



Output Voltage of the Voltage Divider is given by,

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2}$$

Power dissipation through the resistors is given by,

$$P = \frac{V_{in}^2}{R_1 + R_2}$$

Part 01:

Design a Python program to select a pair of resistors with **minimal power dissipation** for a voltage divider circuit to achieve a specified output voltage within a given tolerance range. The program should read input parameters from an 'input.txt' file and write the selected resistor pair separated by a comma to 'output.txt' file. The first line contains the input voltage (V_{in}), desired output voltage (V_{out}), and tolerance (t) separated by commas. The second line contains a comma-separated list of resistor values (in ohms). V_{in} , V_{out} , and resistor values are integers, and the tolerance (t) is a float.

Input File:

```
12, 5, 0.1
110, 32, 204, 124, 72, 176, 500, 80, 150, 91
```

Output File:

```
204, 150
```

Note: Output order should be R1, R2. (Refer the given diagram)

Explanation:

In this example, you should find the resistor pair from the resistors in the second line that produces an output voltage in the $[5 \pm 0.1]$ V range for the input voltage 12 V with minimal

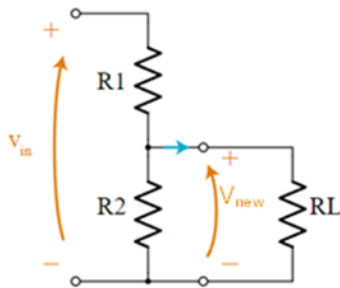
power dissipation.

$R_1 = 110, R_2 = 80, V_o = 5.052632, P = 0.757895$
 $R_1 = 204, R_2 = 150, V_o = 5.084746, P = 0.406780$
 $R_1 = 124, R_2 = 91, V_o = 5.079070, P = 0.669767$
 $R_1 = 176, R_2 = 124, V_o = 4.960000, P = 0.480000$
 $R_1 = 150, R_2 = 110, V_o = 5.076923, P = 0.553846$

The above pairs of resistors produce V_{out} in [4.9, 5.1] range. From these pairs, $R_1 = 204$, and $R_2 = 150$ has the minimal power dissipation.

Part 02:

When you connect a load (R_L) to the output of a voltage divider, the output voltage can change because the load resistor (R_L) is effectively in parallel with the resistor (R_2) in the voltage divider circuit.



New Output Voltage (V_{new}) is given by,

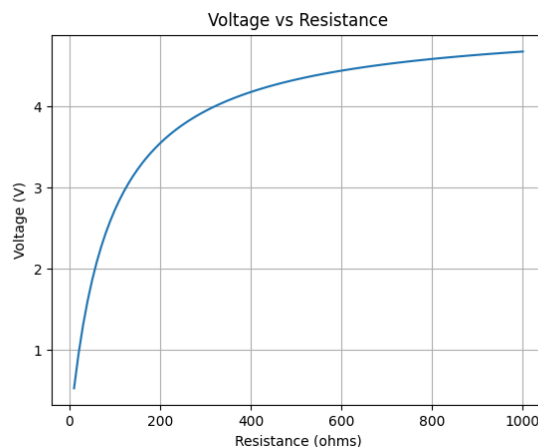
$$V_{new} = V_{in} \cdot \frac{R_2 \parallel R_L}{R_1 + (R_2 \parallel R_L)}$$

Where,

$$R_2 \parallel R_L = \frac{R_2 \cdot R_L}{R_2 + R_L}$$

You need to generate a plot showing how the output voltage (V_{new}) varies with load resistance (R_L) from 10 to 1000 ohms. (Use a step size of 10 ohms)

Example plot for the above input:



Exercise L9.E2 – You are provided with a file named contamination_analysis.txt. This file contains details regarding a contamination analysis (The process of finding unwanted chemical compounds in a certain material) carried out for a set of chemicals by a lab.

A set of chemical names and the molecular structure of the contaminant in each chemical is given in the file.

Example input file (contamination_analysis.txt) :

ABC/D/12	C6-H5-Cl3-Na
DE/W/45	C7-S-O4-Na
MLI/85/9	C6-H6-S-O3-Mg
QT/36/X	C4-Cl3-O3-Cl3-O2
AL/2T/B	C2-N-H4-Na
PP/L2/28	C6-S-O8-Na-H2-Cl3

- Each line of the input file has a chemical name and the molecular formula of the contaminant found in the chemical (ABC/D/12 is the chemical name, and C6-H5-Cl3-Na is the molecular formula for the contaminant in the chemical ABC/D/12).
- The number of chemicals in an input file can vary. The example input file has details regarding 6 chemicals but the input files in test cases can have up to 10 chemicals in a file.

You need to categorize each chemical in the input files based on the contaminant in the chemical. The following criteria should be followed for categorizing chemicals.

Criteria:

1. The chemical is categorized as 'Level_1' if the contaminant contains;
 - a. One 'S' molecule and,
 - b. Four 'O' molecules and,
 - c. One 'Na' molecule
2. The chemical is categorized as 'Level_2' if the contaminant contains;
 - a. One 'S' molecule and,
 - b. Three 'O' molecules and,
 - c. One 'Mg' molecule
3. The chemical is categorized as 'Level_3' if the contaminant contains;
 - a. Two 'O' molecules and,
 - b. Three 'Cl' molecules
4. If any contaminant belongs to more than one category above then the chemical is categorized as 'Level_4'.
5. If the contaminant doesn't belong to any of the above categories then the chemical is categorized as 'Level_0.txt'.

Note : When categorizing, give priority to the order of the criteria.

Ex: If the contaminant has (C6-S-O3-Na-O2-Mg)

One 'S'
Six 'C' s
Five 'O' s
One 'Na'
One 'Mg'

Then the category is Level_1. Because after we categorized it to Level_1 the left molecules are

Six 'C' s
One 'O'

One 'Mg'

The leftover molecules cannot form Level_2.

Instructions : Write a python programme to categorize each chemical according to the given criteria.

- Read the data from the input file *contamination_analysis.txt*.
- Write a function to categorize each chemical in the input data according to the given criteria.
- For each category, create a file named Level_0.txt, Level_1.txt, Level_2.txt, Level_3.txt, and Level_4.txt.
- Add the categorized chemical names to the respective file. *If there is no chemical under a certain category then output an empty file.*
- You need to output **five files** named; Level_0.txt, Level_1.txt, Level_2.txt, Level_3.txt, Level_4.txt as the final output.

Hint: Recommended to use regex patterns (import re) for the implementation.

You should output five files as the final output.

Example output :

Level_0.txt	Level_1.txt	Level_2.txt	Level_3.txt	Level_4.txt
ABC/D/2 AL/2T/B	DE/W/45	MLI/85/9	QT/36/X	PP/L2/28